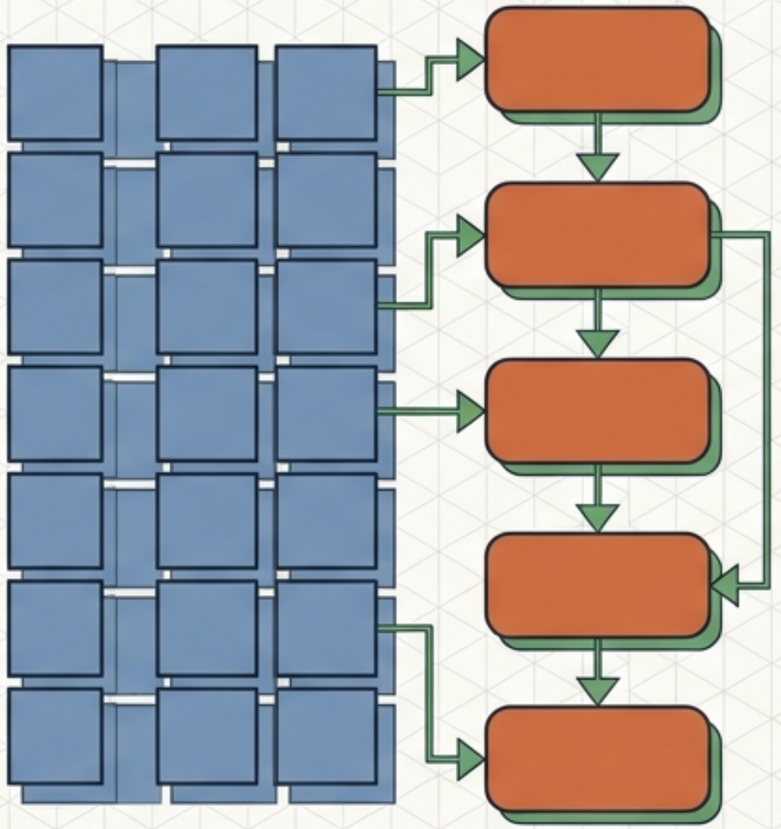
Arsitektur Struktur Data: Dari Array hingga Manipulasi Linked List Kompleks

>_ Panduan visual eksekusi memori, pewarisan OOP, dan algoritma pointer.



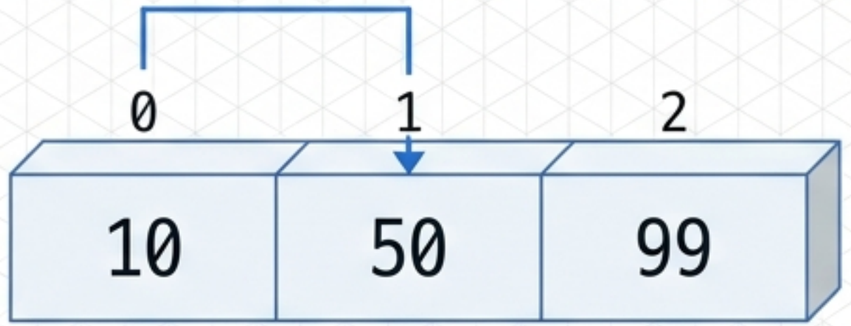
Gemini Notebook

Fondasi 1: Array (Memori Statis & Berurutan)

Array menyimpan data dalam blok memori yang bersebelahan. Ukurannya tetap, membuat akses elemen (indeks) sangat cepat O(1), namun tidak fleksibel untuk penambahan data dinamis.

```
int[] arr = {10, 50, 99}; // Inisialisasi
for(int i = 0; i < arr.length; i++) {
   System.out.println("Indeks " + i + ": " + arr[i]);
}

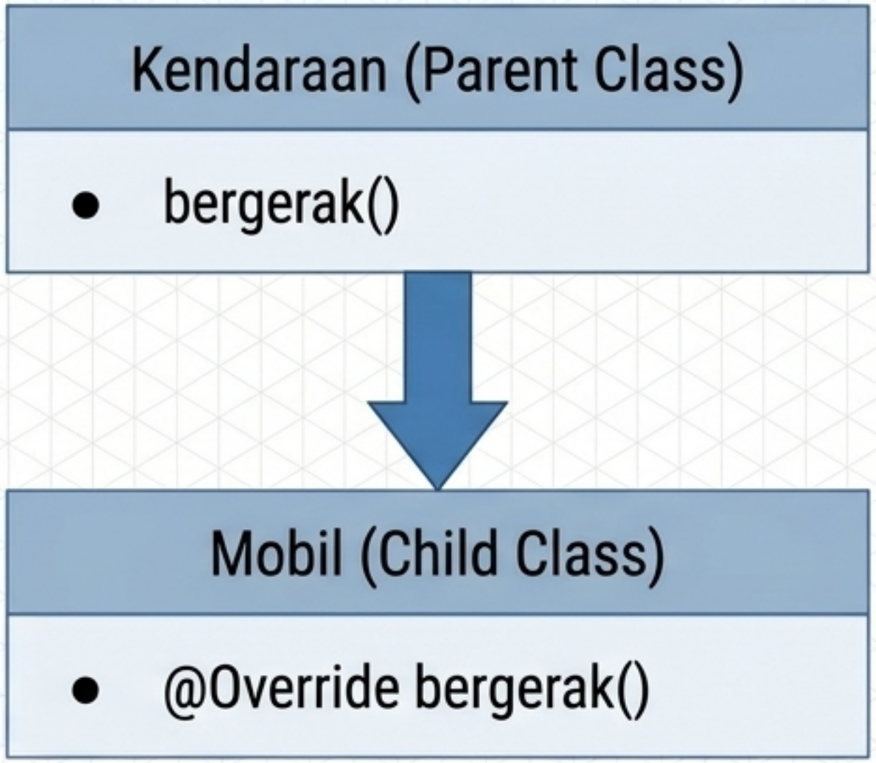
// Output: Indeks 1: 50
```



Gemini Notebook

Fondasi 2: Pewarisan OOP (Cetak Biru Objek)

Pewarisan (Inheritance) memungkinkan sebuah kelas (Child) mewarisi atribut dan metode dari kelas lain (Parent). Ini adalah fondasi dari pembuatan struktur data berbasis objek seperti Node.




```
class Kendaraan {
   void bergerak() { System.out.println("Bergerak maju"); }
}

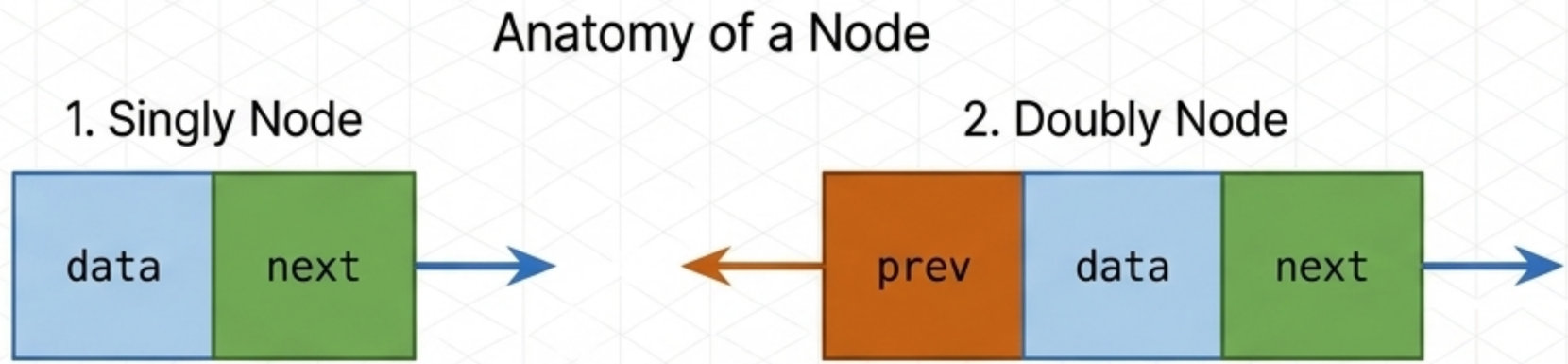
class Mobil extends Kendaraan {
   @Override
   void bergerak() { System.out.println("Mobil melaju kencang!");
}
```

```
Mobil m = new Mobil();
m.bergerak();
// Output: Mobil melaju kencang!
```

Gemini Notebook

Transisi: Mengubah Objek Menjadi Rantai Dinamis

Linked List bukan struktur bawaan seperti Array. Ia adalah sekumpulan Objek (Node) yang saling mengikat menggunakan referensi memori (**pointer**).




```
class Node {
    int data;
    Node next;  // Pointer ke Node berikutnya
    Node prev;  // Pointer ke Node sebelumnya (Doubly)

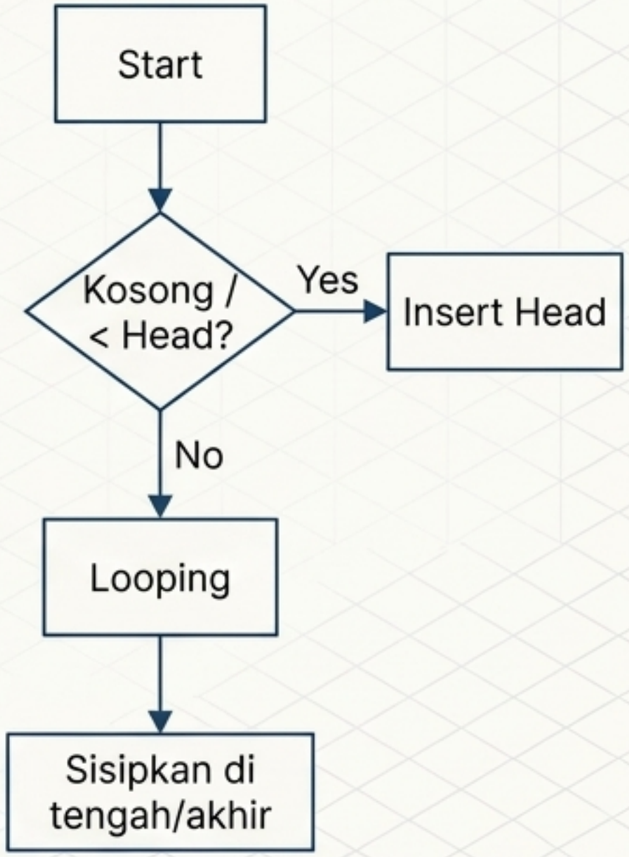
    Node(int data) { this.data = data; }
}
```

Gemini Notebook

Soal 1: Insert into a Sorted Singly Linked List (Kode)

Algoritma menyisipkan x ke dalam list yang sudah terurut ascending. Jika x sudah ada, sisipkan setelah node terakhir dengan nilai yang sama.

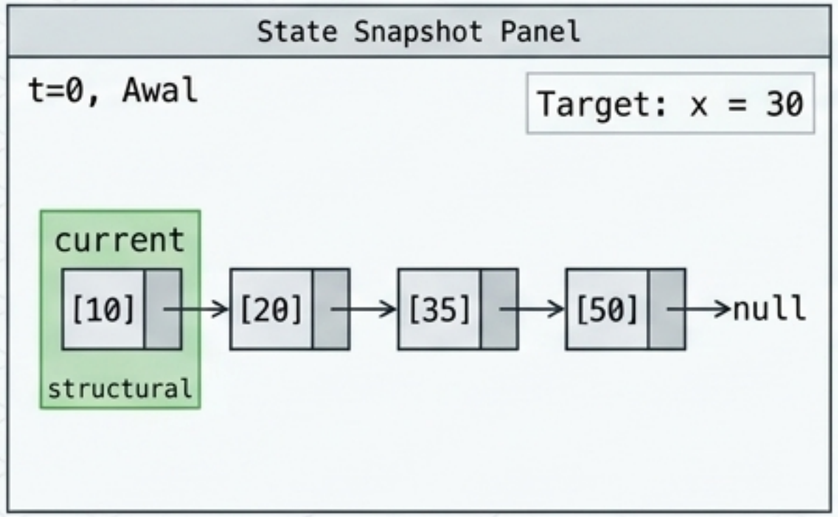
```
Node insertSorted(Node head, int x) {
    Node newNode = new Node(x);
    // Case 1: List kosong atau x lebih kecil dari head
    if (head == null || head.data > x) {
        newNode.next = head;
        return newNode;
    }
    // Case 2: Traversing untuk mencari posisi
    Node current = head;
    while (current.next != null && current.next.data <= x) {
        current = current.next;
    }
    // Menyisipkan node
    newNode.next = current.next;
    current.next = newNode;
    return head;
}
```

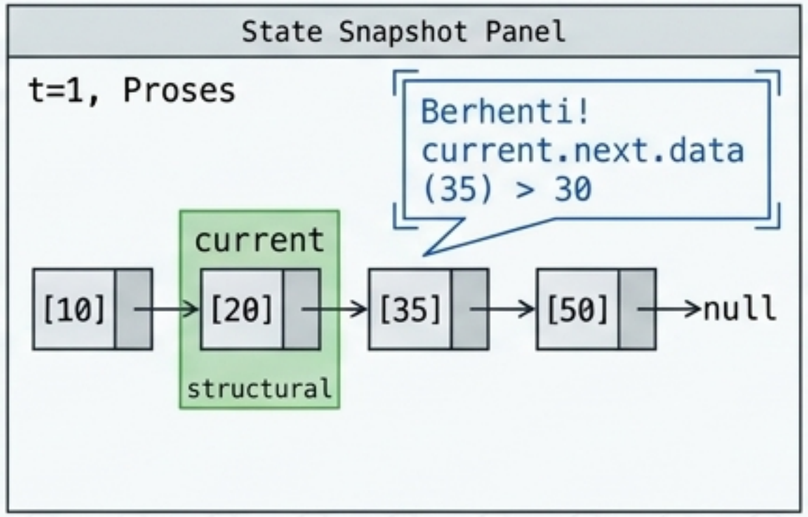


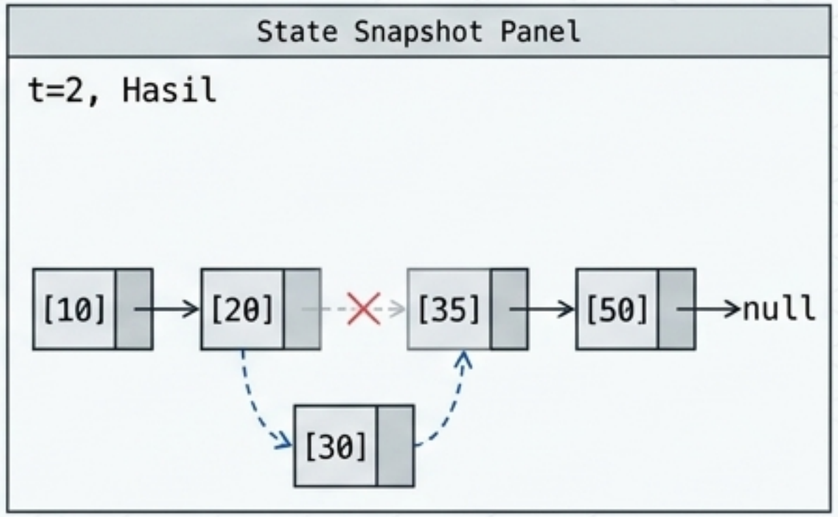
Gemini Notebook

Trace Soal 1: Eksekusi & Pemutusan Pointer

Iterasi berhenti sebelum elemen yang lebih besar dari x. Pointer next dari node current diputus dan dialihkan ke node baru, menjaga urutan data tetap stabil (ascending).







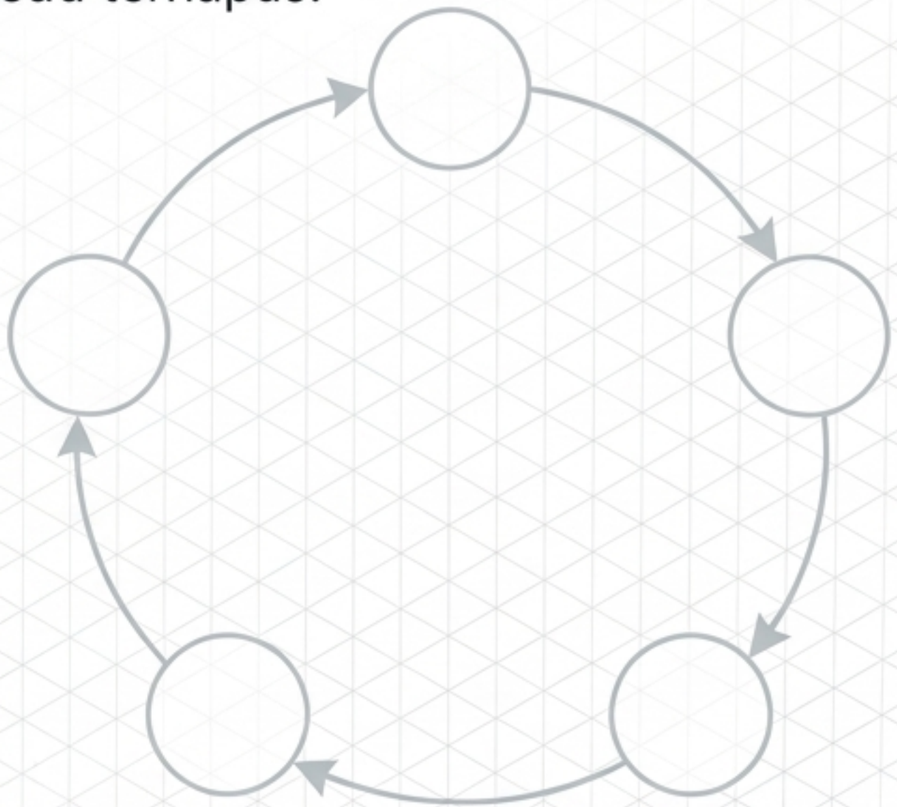
Variable Watch	
current.data	current.next.data
10	20
20	35

Gemini Notebook

Soal 2: Delete All Occurrences in a Circular List (Kode)

Menghapus semua node bernilai x. Tantangan utama struktur sirkular: Mencegah infinite loop saat traversal, dan menangani kasus khusus jika node head terhapus.

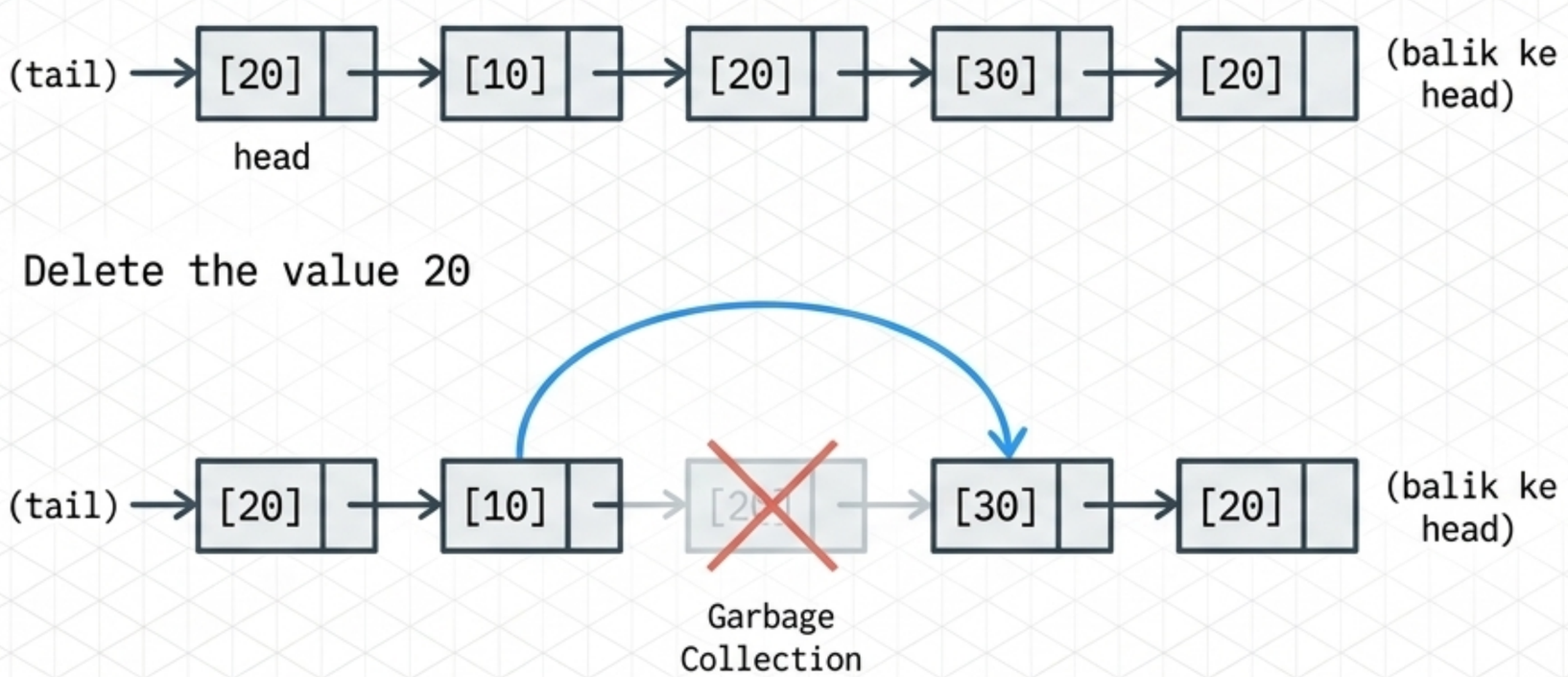
```
Node deleteAllCircular(Node head, int x) {
    if (head == null) return null;
    Node current = head;
    // Hapus node di tengah/akhir
    do {
        if (current.next.data == x) {
            current.next = current.next.next; // Bypass node
        } else {
            current = current.next;
        }
    } while (current.next != head);
    // Hapus head jika nilainya x
    if (head.data == x) {
        if (head.next == head) return null; // Hanya 1 node
        current.next = head.next;
        head = head.next;
    }
    return head;
}
```



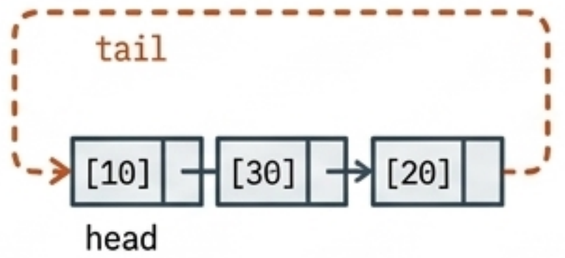
Gemini Notebook

Trace Soal 2: Menangani Head dan Orphan Nodes

Menghapus di Linked List berarti mem-bypass node. Node tanpa pointer akan otomatis dihapus oleh Garbage Collector. Perhatikan bagaimana ekor (tail) memperbarui arahnya ke head yang baru.



Edge Case Callout Box



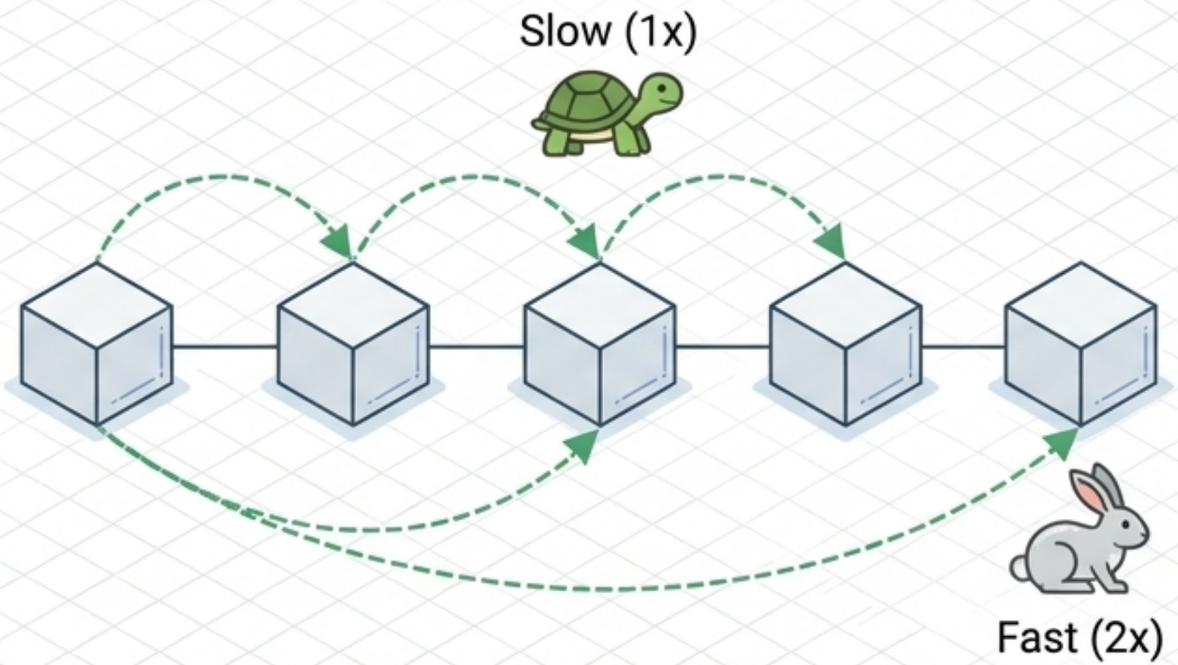
Kasus Khusus: Head Terhapus. Pointer Tail harus diperbarui untuk menunjuk ke head yang baru.

Gemini Notebook

Soal 3: Split Circular List (Algoritma Slow & Fast Pointer)

Membagi satu sirkular list menjadi dua tanpa menghitung total node. Menggunakan dua pointer: slow bergerak 1 langkah, fast bergerak 2 langkah. Saat fast mencapai akhir, slow berada tepat di tengah.

Race Track Metaphor



Java Implementation (Code Block)

```
void splitCircular(Node head) {
    if (head == null) return;
    Node slow = head, fast = head;
    // Menemukan titik tengah
    while (fast.next != head && fast.next.next != head) {
        slow = slow.next;
        fast = fast.next.next;
    }
    Node head1 = head;
    Node head2 = slow.next;

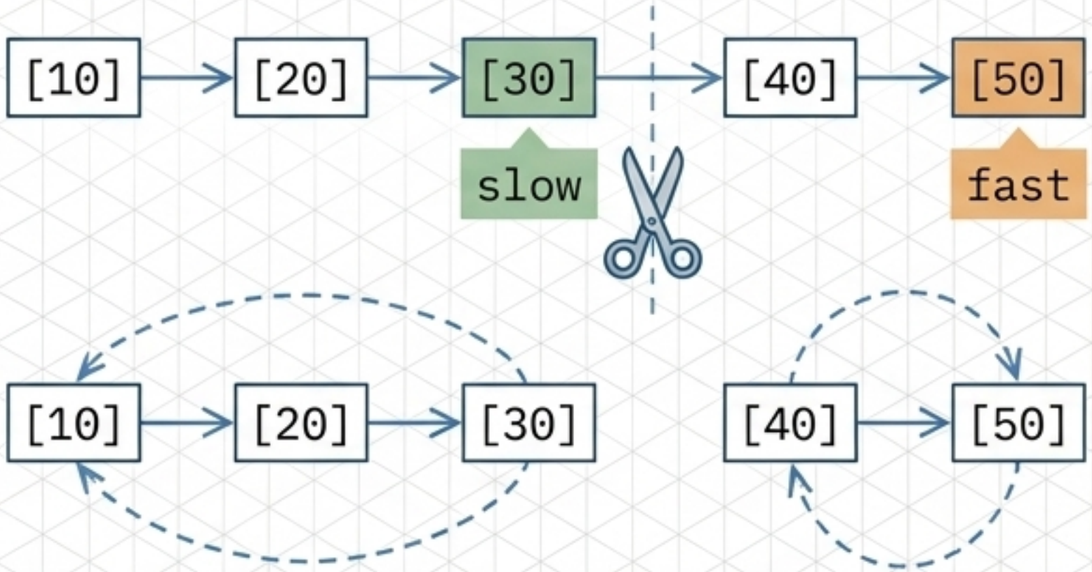
    // Menutup sirkular list kedua
    if (fast.next == head) fast.next = head2; // ganjil
    else fast.next.next = head2;              // genap

    // Menutup sirkular list pertama
    slow.next = head1;
}
```

Gemini Notebook

Trace Soal 3: Eksekusi Pembelahan (Kasus Ganjil)

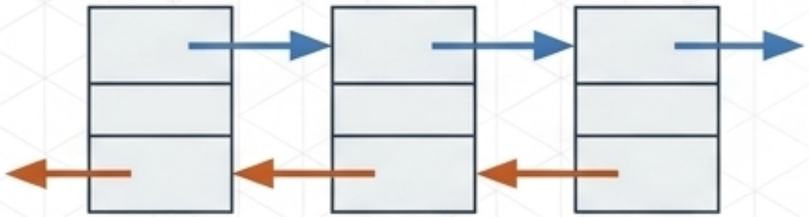
Perhatikan saat fast mencapai node terakhir (50), slow berada tepat di node (30). List dipotong setelah slow, dan kita merajut ulang referensi ekor masing-masing bagian kembali ke head barunya.



Iterasi	Posisi Slow	Posisi Fast
0	10	10
1	20	30
2	30	50 (Berhenti!)

Gemini Notebook

Soal 4: Delete All Occurrences (Doubly Linked List)



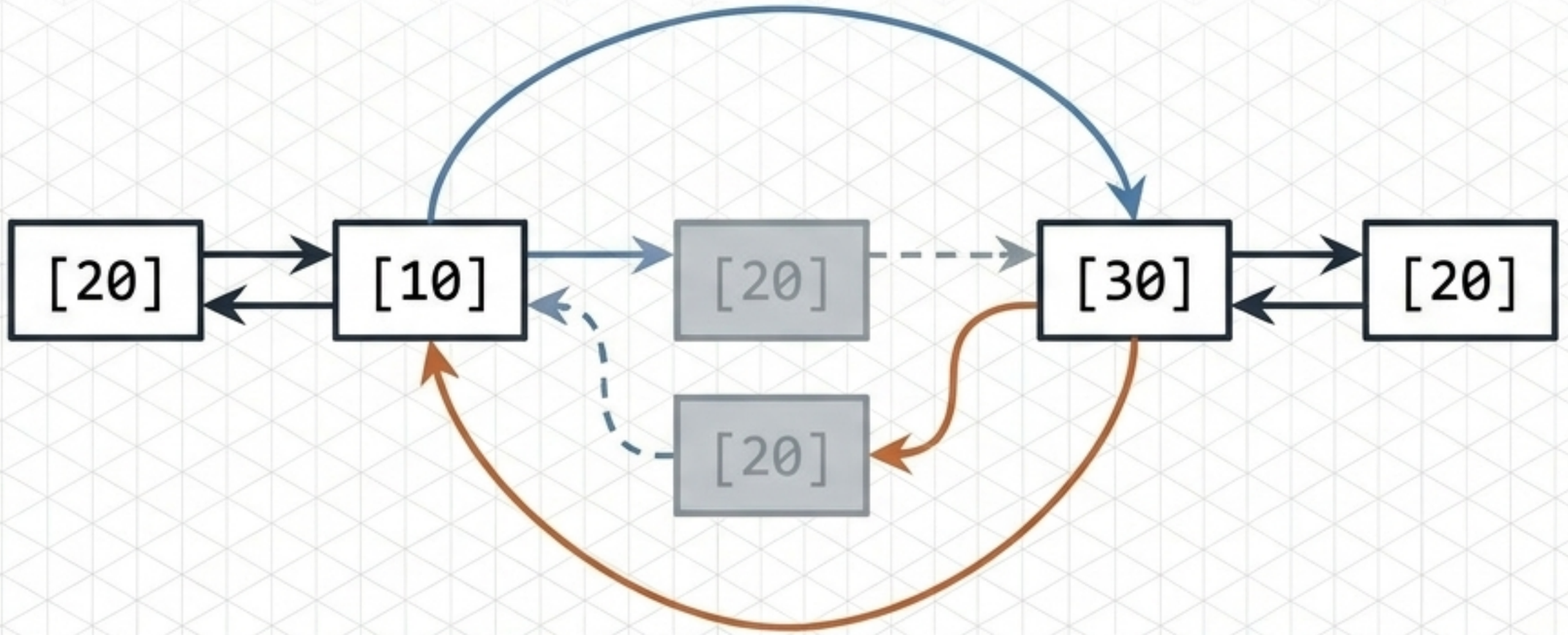
Pada Doubly Linked List, setiap penghapusan mensyaratkan dua operasi: memperbarui pointer next dari node sebelumnya, dan memperbarui pointer prev dari node sesudahnya.

```
Node deleteAllDoubly(Node head, int x) {
    Node current = head;
    while (current != null) {
        if (current.data == x) {
            if (current.prev != null)
                current.prev.next = current.next;
            else head = current.next; // Jika hapus head
            if (current.next != null)
                current.next.prev = current.prev;
        }
        current = current.next;
    }
    return head;
}
```

Gemini Notebook

Trace Soal 4: Operasi Bedah Dua Arah

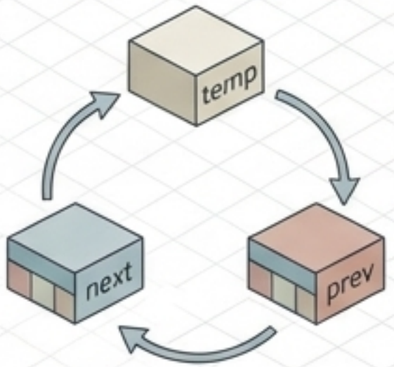
Menghapus berturut-turut atau di ujung mensyaratkan validasi ketat. Jika `current.prev` adalah null, kita menghapus head. Jika `current.next` adalah null, kita menghapus tail.



Edge Case Alert:
Jika node terakhir dihapus, jangan lupa `tail` harus menunjuk ke node sebelumnya yang tersisa.

Gemini Notebook

Soal 5: **Reverse Doubly Linked List In-Place** (Kode)



Membalikkan **list tanpa menukar nilai data** (menghemat memori). Algoritma ini hanya membalik arah panah next dan prev pada setiap node, lalu menukar identitas head dan tail.

```
Node reverseDoubly(Node head) {
    Node current = head;
    Node temp = null;

    // Menukar next dan prev untuk semua node
    while (current != null) {
        temp = current.prev;
        current.prev = current.next;
        current.next = temp;
        // Pindah ke node 'berikutnya' (yang sekarang di prev)
        current = current.prev;
    }
    // Update head baru
    if (temp != null) {
        head = temp.prev;
    }
    return head;
}
```

Gemini Notebook

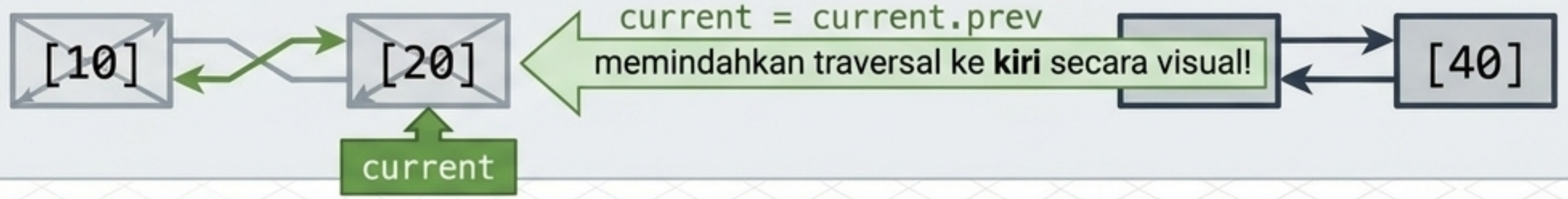
Trace Soal 5: **Balikkan Arah Pointer!**

Hati-hati: Setelah kita menukar `next` dan `prev` pada suatu node, untuk melanjutkan iterasi ke node berikutnya dalam urutan aslinya, kita harus memanggil `current = current.prev`. Ini jebakan logika terbesar!

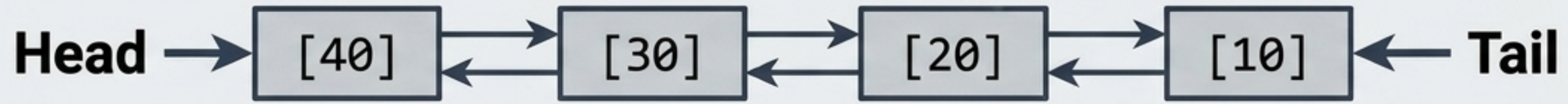
Step 1: Awal



Step 2: Proses Transisi



Step 3: Hasil Akhir



Gemini Notebook

Matriks Sintesis: **Kapan Menggunakan Apa?**

Pemilihan struktur data adalah kompromi antara kecepatan akses dan fleksibilitas modifikasi.

Struktur Data	Akses (Search)	Insert/Delete	Karakteristik & Use Case
Array	O(1) ✓	O(N)	Data statis, ukuran tetap. Cocok: Akses indeks acak cepat.
Singly Linked List	O(N)	O(1) ✓	Satu arah, memori efisien. Cocok: Implementasi Stack dinamis.
Circular Linked List	O(N)	O(1)	Tanpa ujung (tail -> head). Cocok: Round-robin scheduling.
Doubly Linked List	O(N) (Dua Arah)	O(1)	Boros memori (+1 pointer). Cocok: Navigasi riwayat mundur, Cache LRU.

Gemini Notebook